
Geometric algorithms on Meshes in C++ with *Qt*.

Benoît Piranda

1 Introduction

1.1 The initial code

Download the file "*QtCircumcircle.zip*", unzip it and open QtCreator from the ".pro" file. The program allows to visualise a default set of triangles ([Ctrl]+N) or a set defined in a JSON file ([Ctrl]+O). It uses a **Triangle** class that store 3 pointers on a list of vertices (**QVector<Vector2D>**), the colored brush to draw it, and the circumcircle center and radius.

1.2 The interface

The interface is based on the definition of a **MainWindow** containing an instance of **Canvas** that stores and draws all the triangles. The main menu consists of:

File/New Create a simple set of 3 triangles connecting 5 vertices.

File/Open Propose to load a mesh coded in a ".json" file. Two examples are available in the "models" directory.

Show/Circles Toggle the drawing of the circumcircles.

Show/Centers Toggle the drawing of the circumcenters.

Show/Triangles Toggle the drawing of the triangles.

Help Information about the software and the author.

2 Exercises

Exercise 1: Interactions with the mesh

This exercise allows you to manipulate data in the Canvas class by moving vertices, adding triangles by adding a point, or removing triangles by removing a point.

Question 1: Add the processing that detects whether the mouse pointer is close to a point (less than **const int pointSize= 20** pixels), and change the cursor in that case. **setCursor(Qt::Pointi**

Question 2: Add code that allows the vertex to be moved when clicking in the area near a point.

1. Select the point to move and the relative position between the cursor and the point.
2. While the button is down, move the point to follow the mouse.

3. Set the new position of the button when the mouse is released.

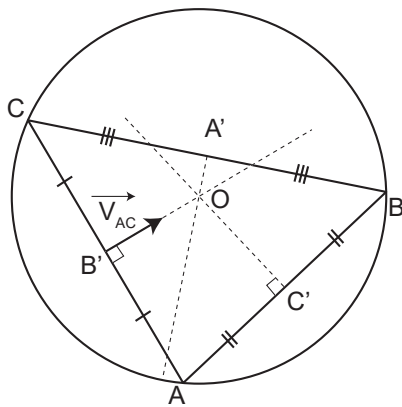
Question 3: What are the conditions for the motion of a point? Add rules to your code to preserve the integrity of the set of triangles.

Question 4: Insert a new point at the position of the mouse pointer using the "Insert" key (if it is not close to another point). If the mouse pointer is inside an existing triangle, explain how to replace it with three new triangles. Implement this functionality in your code.

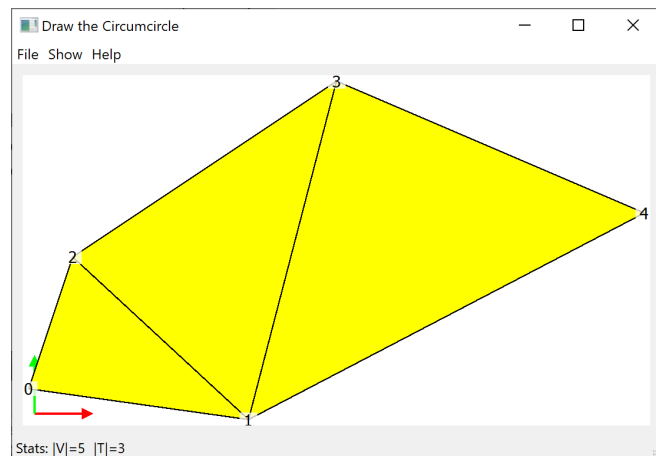
Question 5: If the mouse pointer is outside an existing triangle, explain how to create a new triangle with the shortest edge of the mesh. Implement this functionality in your code.

Question 6: Pressing the 'Delete' key when the mouse pointer is close to a point allows you to delete it. What effect does this have on the mesh? Implement this functionality in your code.

Question 7: Add the save menu entry and propose to save your new mesh in a new json file.



(a) Triangle circumcircle construction.



(b) Initial program window.

Figure 1: Circumcircle illustrations

Exercise 2: Circumcircle of a triangle

This exercise proposes to calculate and draw the circumcircle of a given triangle. A circumcircle is defined by its center O and its radius $OA = OB = OC$. The Figure 1a shows the propriety of the circumcircle in a triangle ABC .

Download the [example code](#)¹ and answer the following questions. The program allows to visualise a default set of triangles ([Ctrl]+N) or a set defined in a JSON file ([Ctrl]+O). It uses a **Triangle** class that store 3 pointers on a list of vertices (**QVector<Vector2D>**), the colored brush to draw it, and the circumcircle center and radius.

Question 1: Demonstrate that $\vec{AO} \cdot \vec{AB} = \frac{1}{2}AB^2$.

Question 2: What are the coordinates of the vector $\vec{V}_{AC}$, orthogonal to $\vec{AC}$ (and oriented to the right of $\vec{AC}$)?

Question 3: Write an equation that places O on the abscissa k on the ray $[B', \vec{V}_{AC}]$.

Question 4: Write an expression of $\vec{AO}$ using $\vec{B'O}$, then using the previous expression show that the expression $k = \frac{AB^2 - \vec{AC} \cdot \vec{AB}}{2 \times \vec{V}_{AC} \cdot \vec{AB}}$ gives the position of O on the ray $[B', \vec{V}_{AC}]$.

Question 5: Express the coordinates of the circumcentre O and the circumradius R from $\{A, B, C\}$.

¹Initial code: https://benoitpiranda.fr/assets/courses/adv_algo/QtCircumcircle.zip

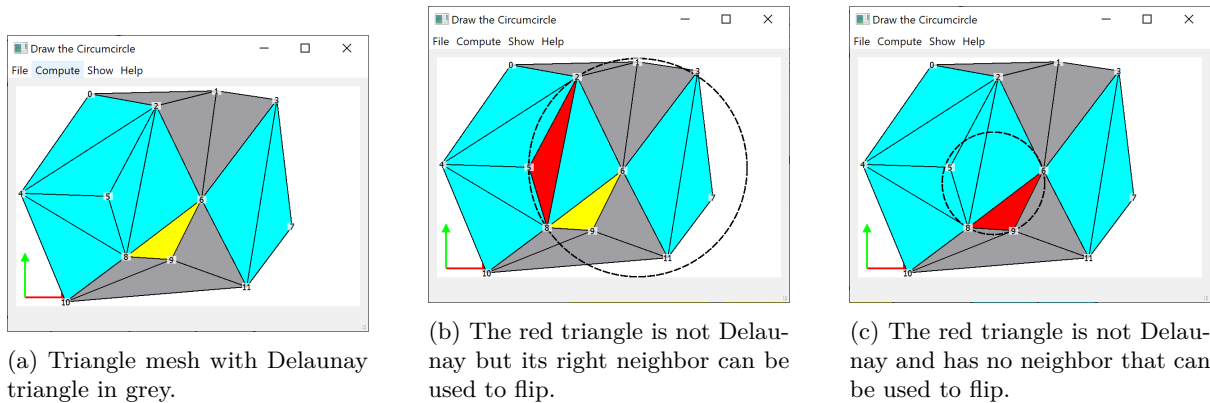


Figure 2: Mesh triangles that in the three Delaunay states.

Question 6: Update the `calculateCircle()` function in the code to compute the position and radius of the circle and verify your values in activating the drawing of the circles.

Exercise 3: Delaunay triangulation

This exercise will complete the previous one in converting the previous mesh into a Delaunay mesh.

Definition 1. A *Delaunay triangulation* (also known as a *Delone triangulation*) for a given set P of discrete points in a plane is a triangulation $DT(P)$ such that no point in P is inside the circumcircle of any triangle in $DT(P)$.

Question 1: For the following experiments we will use one of the meshes described in a JSON format placed in the `models` directory. Update the code to load automatically the file `mesh2.json` on start.

Question 2: Create the function `bool checkDelaunay()` that set properly the `isDelaunay` variable of each triangle according to the definition 1 and returns true if all triangles are Delaunay compliant. You may download the [proposed code](#)² code allowing to calculate the determinant of a 3×3 matrix.

What is the complexity of your function?

Question 3: Add a entry `Compute` in the menu bar and a button `Delaunay` in this menu (with `[Ctrl]+d` for shortcut). The `checkDelaunay()` function must be run when selecting this menu.

Question 4: We can consider three cases presented Figure 2:

1. The triangle is Delaunay, it is colored in grey.
2. The triangle is not Delaunay and it exists a neighbor triangle (sharing an edge) which admit its third vertex inside the circumcenter. These two triangles can be "flipped", color the triangle in cyan.
3. The triangle is not Delaunay and can not be flipped, color it in yellow.

Question 5: Write a code which realize the flip of a triangle and its associated neighbor (if they do not follow the Delaunay rule) when clicking on it. Figure 3 shows several steps to manually transform the mesh to a Delaunay mesh.

Question 6: Create an algorithm that automatically convert the initial mesh into a Delaunay mesh. This treatment is activated by a new submenu `Solve` (with `[Ctrl]+s` for shortcut) in the menu `Compute`. What is the complexity of your algorithm?

²https://benoitpiranda.fr/assets/courses/adv_algo/determinant.zip

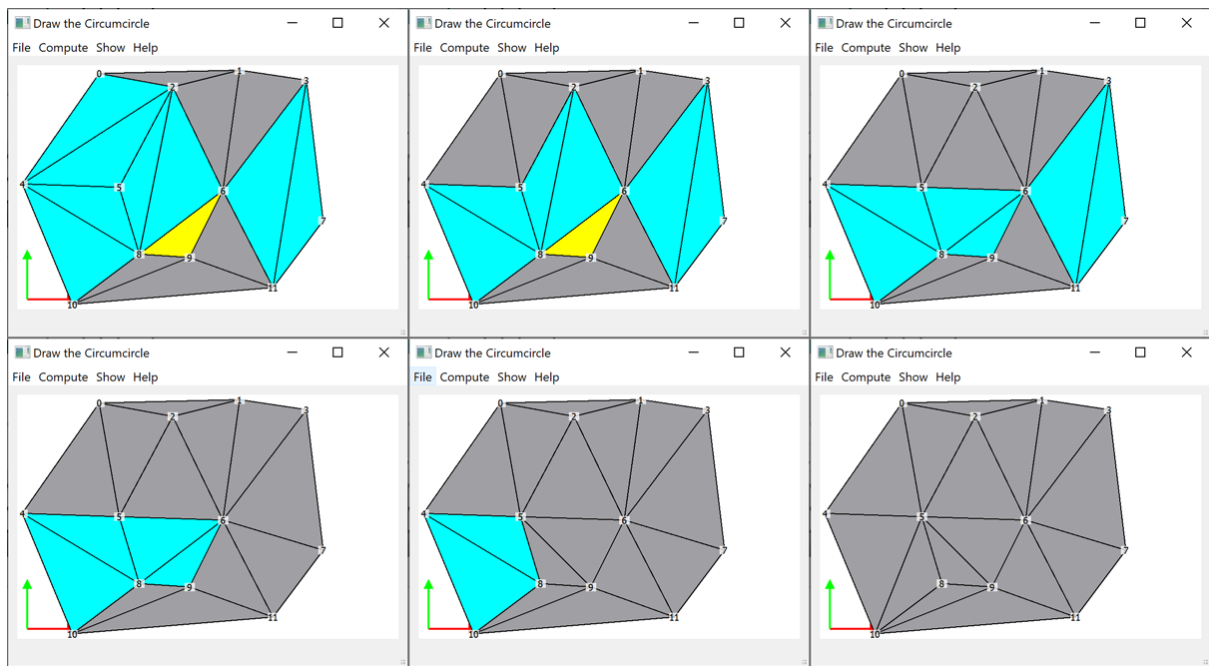


Figure 3: Steps to manually convert the proposed mesh to a Delaunay mesh.